



NEVER MISS BUS

Secure School Bus Live-Tracking Application

SRBS INTERNATIONAL SCHOOL

Android + iOS | Flutter + Firebase

Student · Driver · Admin

Project Documentation — v1.0 · August 2026

1. App Overview

Never Miss Bus is a secure school transportation management app for SRBS International School, built for Android and iOS with Flutter and Firebase. Students see the full live location of their assigned school bus, its route, designated stops and approximate arrival time, and receive important bus notifications. Drivers share GPS with a single tap. School admins manage the complete transport structure.

This is not a generic GPS tracker. Every piece of data is role-fenced on the backend: a student can only ever see their own assigned bus — enforced by server-side security rules, not the app UI.

Key facts

Item	Detail
App name	Never Miss Bus
School	SRBS International School
Platforms	Android + iOS (single Flutter codebase)
Frontend	Flutter · Riverpod state management · go_router navigation
Backend	Firebase — Auth, Cloud Firestore, Realtime Database, Cloud Functions, Cloud Messaging (FCM)
Maps	Google Maps (google_maps_flutter) with a custom school-bus marker
Roles	Student · Driver · Admin — separate permissions and interfaces
Quality	flutter analyze: 0 issues · 15/15 unit tests passing · strict-TypeScript Cloud Functions

2. How Live Tracking Works

DRIVER GPS → SECURE BACKEND → REAL-TIME DATABASE → AUTHORIZED STUDENT → LIVE MAP

- Driver taps **Start Trip** — the phone begins sending a GPS fix every ~5 seconds / 20 metres through a foreground service (keeps working with the screen off; no interaction needed while driving).
- Each fix is written to a single per-bus node in Firebase Realtime Database with a **server timestamp**, so freshness cannot be faked.
- Students subscribed to their assigned bus see the marker move live, with the route line, all stops, their own stop highlighted with a star, and the school location.
- Driver taps **End Trip** — tracking stops and the location node is deleted. If the driver's phone dies mid-trip, a server job auto-closes the trip within 15 minutes and notifies students that tracking is unavailable.

Honest status policy

Data age	What the student sees
≤ 30 seconds	LIVE — pulsing green badge, full-opacity bus marker
30 s – 3 min	“Last updated X minutes ago” — amber badge, dimmed marker
> 3 min / no trip	“Location temporarily unavailable” or “No trip in progress” — no bus marker at all

ETA is always worded “Bus arriving in approximately N minutes” and is hidden entirely whenever the data is not reliable — an estimate is never presented as a guarantee.

3. The Three Roles

3.1 Student

- Secure school-provided login (accounts created only by the school admin).
- Home: friendly greeting, assigned bus card (number, plate, ON TRIP / IDLE), live tracking status, big **Track My Bus** button, assigned stop, quick actions and latest alert.
- Live Map: bus marker, route, stops, my stop (star), school, “Center on Bus” control.
- Bus & Route details, ordered stop timeline, alerts inbox, profile with school contact info, settings, logout confirmation.
- Cannot select another bus, change assignments, or see any other student's data — blocked by backend rules.

3.2 Driver

- Separate secure account, assigned to one bus by the admin.
- One giant **START TRIP / END TRIP** button — chooses morning pickup or afternoon drop, then everything is automatic.
- Status tiles: GPS on/off, internet connected/offline, sharing live/stopped.
- Route stop list in large glanceable text; trip resumes automatically if the phone restarts mid-route.
- Can only control their own bus and trip — enforced server-side.

3.3 Admin

- Dashboard with fleet stats and active trips.
- Manage students, drivers, buses, routes and ordered bus stops.
- Assign students → buses → stops; assign drivers → buses.
- Live monitoring map of all active buses at once.
- Send announcements to everyone or to a single bus.
- Enable/disable accounts (takes effect within minutes via token revocation) and review a tamper-proof activity/audit log.

4. Screen Inventory

Student (10)	Driver (9)	Admin (16 capabilities)
Splash · Login · Home · Live Map · Bus/Route Details · Bus Stops · Alerts · Profile · Settings · Logout confirm	Login · Home/Dashboard · Assigned Bus · Start Trip · Active Trip · Route/Stops · GPS & Connection Status · End Trip confirm · Profile	Login · Dashboard · Students · Student details · Drivers · Driver details · Buses · Bus details · Routes · Stops · Assignments · Live Monitoring · Notifications · Access Management · Audit Logs · Settings

All three interfaces share one design system: rounded cards with soft shadows, school-blue + bus-amber palette (WCAG AA contrast), large readable typography, friendly icons, large touch targets, consistent status pills, dialogs, snackbars, loading / empty / error / offline states.

5. Architecture

Why two databases?

- **Cloud Firestore** — structured, queryable data: users, buses, routes, stops, trips, notifications, audit logs.
- **Realtime Database** — one tiny node per bus for high-frequency GPS (a ping every 5 seconds would be wasteful in Firestore). The node is deleted when the trip ends.

Flutter app layering

features (UI) → providers (Riverpod streams) → services (Firebase gateways) → models (pure Dart)

- UI never talks to Firebase directly; services are the only layer importing Firebase SDKs.
- go_router with role-based redirects: signed-out users only reach Login; each role is fenced into its own subtree.
- Cloud Functions (TypeScript) hold all privileged logic: account provisioning, assignment changes, notification fan-out, geofence alerts, stale-trip sweeper, audit logging.

Main data collections

Collection	Purpose
users/{uid}	Profile, role mirror, bus/stop assignment, FCM tokens, settings, private inbox subcollection
buses/{busId}	Bus number, plate, capacity, driver, route
routes/{routeId}	Name, bus, ordered stop list, school location, direction
stops/{stopId}	Name, location, order, scheduled time
trips/{tripId}	Active/completed trip records with driver and direction
auditLogs/{id}	Tamper-proof activity trail (written only by Cloud Functions)
RTDB /liveLocations/{busId}	Live GPS fix: lat, lng, heading, speed, server-stamped updatedAt

6. Security & Privacy

Authorization lives in the backend, not the app. Roles and bus assignments are Firebase Auth custom claims, set only by Cloud Functions and enforced by Firestore + Realtime Database security rules. A modified client cannot widen its own access.

Guarantee	How it is enforced
Student can't change own bus	Self-writes limited to FCM tokens, settings and last-seen; assignment fields are Cloud-Function-only
Student can't see other students	User docs readable only by owner or admin; notification inbox is private per user
Student can't track other buses	RTDB read rule requires the bus ID in the student's signed token to match the node being read

Guarantee	How it is enforced
Driver controls only own trip	Trip create/update rules bind driverId and busId to the caller's token claims
No public tracking URLs	Default-deny rules everywhere; zero unauthenticated surface
Fast revocation	Disable/reassign revokes refresh tokens; app force-refreshes on resume
Passwords	Managed by Firebase Authentication (scrypt) — never stored in plain text anywhere
Audit trail	Clients cannot write or delete audit logs; admin read-only
Notification privacy	No FCM topics (any client can join a topic); audiences resolved server-side, sent to explicit device tokens
Secrets hygiene	No API keys in source code: Maps keys in git-ignored files, server credentials only in the Functions runtime

Privacy: only necessary data is collected. Students have no phone numbers in the system; bus location exists only during an active trip and is deleted afterwards; student devices never share their own location — only the driver's phone publishes GPS, with a visible notification.

7. Notifications

Notification	Trigger	Audience
Trip Started	Driver starts a trip	Students of that bus
Bus Approaching	Bus within ~800 m of a stop (server geofence)	Students assigned to that stop
Bus Reached Stop	Bus within ~120 m of a stop	Students assigned to that stop
Bus Reached School	Bus arrives at school on a pickup trip	Students of that bus
Tracking Unavailable	No GPS signal for 15 min (auto-close)	Students of that bus
School Announcement	Sent by admin	Everyone, or one bus

8. What Must Be Configured (no fake keys ship)

- **Firestore project** — create it, enable Email/Password auth, Firestore, Realtime Database, FCM; run `flutterfire configure` to generate the real config files.
- **Deploy the backend** — `firebase deploy --only firestore:rules,firestore:indexes,database,functions`. The rules ARE the access control; never launch without them.
- **First admin account** — one-time bootstrap script with the Admin SDK (documented in docs/SETUP.md); every later account is created inside the app.
- **Google Maps keys** — one Android-restricted and one iOS-restricted key, kept in git-ignored files (android/local.properties and ios/Runner/Secrets.plist).
- **iOS push** — upload an APNs auth key (.p8) in Firebase Console; enable Push Notifications + Background Modes in Xcode.
- **Android release signing** — generate a keystore and wire key.properties before publishing.

Runtime permissions handled in-app

- Location (driver): requested at Start Trip with clear guidance if denied or GPS is off.
- Notifications (all roles): requested on first run; Settings screen offers a fix-it path if declined.

9. Quality & Edge Cases

Situation	App behaviour
No internet	Offline banner; cached data marked as possibly out of date
GPS disabled / permission denied	Clear driver guidance with a one-tap path to system settings
Driver phone dies mid-trip	Server auto-closes the trip in 15 min and notifies students honestly
Bus has no active trip	“No trip in progress — tracking starts when the driver begins the trip”
Expired / revoked session	Automatic bounce to login

Situation	App behaviour
Unauthorized access attempt	Blocked by backend rules regardless of what the client does
Server/database failure	Friendly error views with retry buttons

Verified: flutter analyze — 0 issues · 15/15 unit tests passing (including tests that the app can never show stale GPS as live and never fabricates an ETA) · Cloud Functions compile clean under strict TypeScript.

Full technical references in the project: docs/ARCHITECTURE.md · docs/DATA_MODEL.md · docs/SECURITY.md · docs/SETUP.md